Промт: 1000 требований по улучшению игры и интерфейса (Kotlin / Android)

Готовый промт для AI-ассистента (Gemini в AI Studio, Claude и др.): вставьте документ целиком. Внутри 1000 пронумерованных требований (001–1000) в 31 разделе.

РОЛЬ

Ты — Senior Android-разработчик (Kotlin, Jetpack Compose), UI/UX-дизайнер и геймдизайнер мобильных игр. Пиши рабочий production-код без заглушек и псевдокода, решения объясняй коротко.

КОНТЕКСТ ПРОЕКТА

Игра — коллекционный симулятор генерации автомобильных номеров («слот-машина номеров») на русском языке: игрок тратит ₽ на генерацию, получает номер с редкостью и ценой, продаёт его или собирает коллекцию. Сейчас есть веб-прототип (Google AI Studio); цель — нативная Android-игра на Kotlin с лучшим интерфейсом, анимациями, экономикой и производительностью.

Что есть сейчас (по скриншотам прототипа):

- Нижняя навигация: «Номер», «Авто 4K», «105+ FX», «Кейсы», «ДПС», «Гонки», «Фортуна», «Kotlin APK» — 8 пунктов.
- Экран «Номер»: верхняя панель (меню, чип «105+ FX», чип кэша «512 МБ · Ultra 4K» с зелёной точкой, баланс «20 138 013 ₽»); карточка редкости («ОБЫЧНЫЙ», 5-сегментная шкала, цена «675 ₽»); крупный красный знак «Р 206 УО», регион «7», «RUS»; жёлтая кнопка «СГЕНЕРИРОВАТЬ (30 507 ₽)» с подсказкой «Кликните по номеру или нажмите Пробел»; чип страны и типа «Беларусь · Военный».
- Экран «Меню»: карточка «Генератор» (белый знак `A 777 AA`), событие «Неделя автомобилиста» с таймером «Старт через 0 д 4 ч», «Ежедневные задания» (маски `A***AA` — собрано, `*777**` — в процессе), «HD КЭШ ИГРЫ» на 500 МБ («4K текстуры ГОСТ, 3D автопарк и HD выхлоп», кнопка «Управлять»).
- Заметные недочёты: чип «Беларусь · Военный» не совпадает со знаком «RUS»; подсказка про «Пробел» на сенсорном экране; регион «7» без ведущего нуля; пустые зоны сверху и снизу на высоких экранах; серый текст с низким контрастом; слишком много пунктов в нижней навигации.

ТЕХНИЧЕСКИЕ РАМКИ

- Kotlin (актуальная стабильная версия), Gradle Kotlin DSL + version catalog, minSdk 26, targetSdk — актуальный.
- UI: Jetpack Compose + Material 3 (своя тема), Canvas и `graphicsLayer` для анимаций, Navigation Compose (type-safe routes).
- Архитектура: MVVM + однонаправленный поток данных (UDF/MVI), слои `ui` / `domain` / `data`, Hilt, Coroutines + Flow.
- Данные: Room, DataStore, WorkManager; офлайн-first; сеть (Ktor/Retrofit) — только где нужна.
- Графика и звук: Canvas и AGSL (с запасным путём для API ниже 33), 3D — Filament/SceneView, звук — SoundPool/Oboe + Media3.
- Платформенные сервисы — за интерфейсами, без жёсткой привязки к Google Play Services (Google Play / RuStore / AppGallery).
- Новая зависимость — только с коротким обоснованием.

ПРАВИЛА РАБОТЫ

1. Требования ниже пронумерованы 001–1000. Выполняй их итерациями по 20–40 пунктов; всё сразу в один ответ не помещается.
2. В первом ответе дай план (до 15 строк): модули проекта, порядок блоков, главные риски. Затем сразу реализуй блок 1.
3. Каждый ответ: (а) номера закрытых требований; (б) полный код изменённых файлов с путями, без многоточий и сокращений; (в) как проверить результат; (г) что дальше.
4. При противоречии выбирай вариант с лучшим UX и коротко объясняй выбор. Невыполненный пункт помечай «отложено» с причиной — молча не пропускай.
5. Любая анимация отключаема (Reduce motion) и укладывается в кадр 16,6 мс.
6. Все тексты — в `strings.xml`, размеры и цвета — в токенах дизайн-системы, без «магических» чисел.
7. Пункты с пометкой (опц.) — по возможности, остальные обязательны.
8. Закончив итерацию, напиши «Готово. Продолжить с пункта N?» и жди команды «Дальше».

ПОРЯДОК ИТЕРАЦИЙ

- Фаза 1 — каркас и главный экран: 001–080.

- Фаза 2 — знак, генерация, прокрутка с откидыванием: 081–200.
- Фаза 3 — логика, страны, экономика, коллекция: 201–340.
- Фаза 4 — режимы (кейсы, ДПС, гонки, фортуна, авто): 341–480.
- Фаза 5 — звук, события, прогресс: 481–580.
- Фаза 6 — навигация, дизайн-система, адаптивность: 581–690.
- Фаза 7 — производительность, ресурсы, данные: 691–790.
- Фаза 8 — соцфункции, безопасность, доступность, локализация: 791–890.
- Фаза 9 — онбординг, тесты, релиз, приёмка: 891–1000.

ТРЕБОВАНИЯ (001–1000)

Раздел 1. Архитектура, стек и структура проекта (001–040)

001. Модули: `:app`, `:core:{ui,designsystem,domain,data}`, `:feature:{generator,garage,cases,dps,races,fortune,menu}`.

002. Gradle Kotlin DSL, version catalog (`libs.versions.toml`) и convention-плагины для общих настроек модулей.

003. Build types `debug` / `release` / `benchmark` и flavors по магазинам: `gms`, `rustore`, `appgallery`.

004. Ключи подписи — только из переменных окружения или CI-секретов, не в репозитории.

005. На каждый экран — `UiState`, `UiEvent`, `UiEffect`; единый однонаправленный поток данных.

006. `ViewModel` отдаёт `StateFlow`; в Compose состояние читается через `collectAsStateWithLifecycle()`.

007. UI-состояния неизменяемы: `@Immutable`, `kotlinx.collections.immutable`.

008. Hilt: модули по слоям; диспетчеры, `Clock` и `Random` — через интерфейсы и `@Qualifier`.

009. `:core:domain` — чистый Kotlin без `android.*`; логика в use case-классах (`GeneratePlate`, `EvaluateRarity`, `SellPlate`).

010. Генератор — чистая функция `generate(spec, random): Plate` без побочных эффектов.

011. `Random` и `Clock` инъецируются, поэтому тесты воспроизводимы по seed.

012. Исход случайности записывается в БД до старта анимации; убийство процесса не «перебрасывает» результат.

013. Room: `PlateEntity`, `LedgerEntry`, `CaseEntity`, `TaskEntity`; DAO; схемы экспортируются в репозиторий.

014. DataStore (Proto) для настроек и флагов; схема версионируется.

015. Repository-паттерн; наружу — `Flow` с `distinctUntilChanged()`, без `LiveData`.

016. Structured concurrency: никакого `GlobalScope`; корректная обработка `CancellationException`.

017. Navigation Compose с type-safe маршрутами (`@Serializable`) и вложенными графами для вкладок.

018. Игровой цикл мини-игр: `withFrameNanos`, фиксированный шаг 1/60 с, ограничение `deltaTime`.

019. Единая шина `GameEventBus` (`SharedFlow`): задания, достижения и аналитика слушают одни и те же события.

020. Ошибки — `sealed interface AppError` и `Result`; исключения не используются для управления потоком.

021. `Logger` — интерфейс; в release пишет только ошибки; никаких `println`.

022. `detekt`, `ktlint` и Android Lint в CI; для core-модулей предупреждения считаются ошибками.

023. `explicitApi()` в core-модулях; KDoc у публичного API.

024. Флаги функций и Remote Config — за интерфейсом `FeatureFlags` с локальными значениями по умолчанию.

025. `TimeProvider` (kotlinx-datetime) вместо `System.currentTimeMillis()`; сверка с серверным временем.

026. Строки, цвета и размеры — только ресурсы и токены; никакого хардкода в экранах.

027. Границы модулей проверяются автоматически (Konsist/ArchUnit): domain ничего не знает про Android.

028. Включить Compose compiler reports и `stability_config.conf`; отчёт о нестабильных классах в CI.

029. Состояние переживает убийство процесса: `SavedStateHandle`, `rememberSaveable`, данные в Room.

030. `androidx.startup` и ленивая инициализация тяжёлых компонентов (звук, 3D, аналитика).

031. SplashScreen API, показ не дольше 1 с, без искусственных задержек.

032. R8 full mode, `shrinkResources`, проверенные keep-правила.

033. Нативные библиотеки (Oboe, Filament) совместимы с 16 КБ page size.

034. Поворот экрана, смена темы и языка не сбрасывают анимации и игровое состояние.

035. Диспетчеры инжектируются: UI — `Dispatchers.Main.immediate`, диск — `IO`, расчёты — `Default`.

036. Debug-меню (только debug): выдать ₽, форсировать редкость, сменить время, FPS-оверлей.

037. CI на каждый PR: сборка, unit-тесты, lint, detekt, ktlint; артефакт — debug APK.

038. Документация: `README.md`, `ARCHITECTURE.md`, ADR в `docs/adr/` для каждого значимого решения.

039. Версионирование: semver, `versionCode` из номера сборки CI.

040. Версии зависимостей зафиксированы; Renovate или Dependabot; проверка лицензий зависимостей.

Раздел 2. Главный экран «Номер» (041–080)

041. Пустую зону между верхней панелью и карточкой редкости занять полезным: баннер события и «лента последних находок».

042. Фон-сцена: дорога в перспективе и огни фар с лёгким параллаксом; не отвлекает от знака.

043. Знак и кнопка — единый блок в оптическом центре (≈ 40% высоты); на высоких экранах лишнее место распределяется через `weight`.

044. Знак занимает ⩾ 90% ширины, пропорции 520:112, боковые отступы 16 dp.

045. Кнопка «СГЕНЕРИРОВАТЬ»: состояния idle / pressed / rolling / disabled / нет средств.

046. Нажатие: `scale` 0.96 + ripple + хаптик `CLICK`; возврат пружиной.

047. Нет средств: тряска кнопки (±8 dp, 300 мс), красная подсветка и текст «Не хватает N ₽» с действием.

048. Долгое нажатие на кнопку запускает автогенерацию ×5 / ×10 с лимитом трат.

049. Подсказку про «Пробел» показывать только при физической клавиатуре; на сенсорных экранах — «Нажми на номер».

050. Клавиши: Space и Enter — генерация, Esc — остановка автогенерации.

051. Тап по знаку — генерация; двойной тап — «заморозить» номер.

052. Во время ролла повторные тапы игнорируются (`Mutex`/debounce): никаких двойных списаний.

053. Баланс при изменении анимированно «пересчитывается» (count-up 400–600 мс).

054. Над балансом всплывает дельта «−30 507 ₽» / «+675 ₽»: цвет и знак, исчезает за 1,2 с.

055. Баланс в компактном виде («20,1 млн ₽»), если не влезает; полное значение по тапу; цифры табличные.

056. Тап по балансу открывает «Кошелёк»: операции, капитал, источники и траты.

057. Карточка редкости: 5 сегментов шкалы заполняются по очереди (по 80 мс) с лёгким щелчком.

058. Цвет сегментов и рамки — по редкости; название и цена меняются кросс-фейдом.

059. Тап по карточке — шит «Почему такая цена»: паттерны и множители.

060. Чип «страна · тип» открывает bottom sheet выбора; формат и цвет знака обновляются синхронно.

061. Исправить рассинхрон: чип «Беларусь · Военный» и знак «RUS» должны соответствовать друг другу.

062. Свайп по знаку влево/вправо переключает тип номера; подсказка при первом запуске.

063. Под знаком — действия: «Продать», «В гараж», «Заморозить», «Поделиться».

064. Продажа: знак «улетает» к балансу, монеты, баланс подпрыгивает.

065. Отмена продажи 5 с (Snackbar); проданное лежит в корзине 24 ч.

066. Лента последних 20 генераций (мини-знаки); тап открывает карточку.

067. Мини-статистика: всего генераций, лучшая находка, серия без редких (pity).

068. Мини-баннер активного события с таймером; тап — экран события.

069. Бейджи на вкладках: бесплатный кейс готов, задание выполнено.

070. Чипы верхней панели одной высоты 40 dp; зона нажатия ⩾ 48 dp.

071. Чип «105+ FX» открывает микшер звуков (bottom sheet).

072. Чип кэша «512 МБ · Ultra 4K» открывает «HD-кэш»; цвет индикатора отражает статус (ок / обновление / нет).

073. Кнопка меню — плавный переход (shared element) на экран «Меню».

074. Edge-to-edge: контент под системными барами, отступы через `WindowInsets.safeDrawing`.

075. Нижняя навигация высотой ⩾ 64 dp не перекрывает контент и учитывает gesture bar.

076. Фон: медленно перетекающий градиент; параллакс по гироскопу (опц.).

077. Idle-анимации: «дыхание» кнопки (период 3 с), парение знака ±2 dp; выключаются при Reduce motion.

078. Текущий номер и состояние ролла сохраняются при повороте и убийстве процесса.

079. При первом запуске — skeleton знака вместо пустоты, без мигания.

080. Три контекстные подсказки для новичков (tooltips), каждая показывается один раз.

Раздел 3. Рендер номерного знака (081–120)

081. `PlateView` рисуется на `Canvas` векторно, а не картинками: чёткость на любом размере.

082. API: `PlateView(plate, style, modifier, interactive)` — stateless, без бизнес-логики внутри.

083. Геометрия по ГОСТ Р 50577: 520×112 мм; все размеры — доли ширины знака.

084. Шрифт, близкий к ГОСТ (проверить лицензию), запасной — Roboto Condensed Bold.

085. Допустимые буквы: `A B E K M H O P C T У X`; остальные не генерируются и не рисуются.

086. Текст рисуется через `TextMeasurer` + `drawText`, а не десятком `Text`.

087. Статические слои — в `drawWithCache`; готовые изображения знаков — в `LruCache<ImageBitmap>` по ключу.

088. Рамка: чёрная, радиус ≈ 3–4% высоты, внутренний контур по периметру.

089. Рельеф символов: двойной штрих со смещением и мягкая внутренняя тень.

090. Блок региона: код сверху крупно, под ним `RUS` и флаг; вертикальный разделитель.

091. Регионы 01–09 с ведущим нулём; трёхзначные коды — уменьшенным шрифтом.

092. Флаг — вектор (три полосы) с правильными пропорциями, без bitmap.

093. Для других стран — флаг и код страны по их стандарту (сверить с официальным описанием).

094. Блик: градиентный шейдер, сдвигается по наклону устройства и скроллу (AGSL на API 33+, иначе `Brush`).

095. Два крепёжных отверстия: тонкие, с внутренней тенью.

096. Материал: зерно металла (noise-overlay) и лёгкое отражение.

097. Свечение по редкости (glow): радиус и цвет — из токенов, анимируется при появлении.

098. Рамки номера: хром, карбон, неон, золото — сменный `PlateFrame`.

099. Слой износа (царапины, пыль) — опциональный, для «б/у» номеров.

100. Цвета типов (белый, чёрный, жёлтый, синий, красный) — токены; контраст символов ≥ 7:1.

101. Двухстрочные и квадратные знаки (мото, прицеп) — отдельные layout-ы.

102. Авто-подгонка размера текста под ширину (min/max), без обрезки.

103. Равные межсимвольные интервалы; цифры моноширинные.

104. Подсветка выбранных символов (пульсация) — для разбора редкости.

105. Reveal-маска: символы проявляются по очереди (по 60 мс).

106. 3D-наклон при касании и перетаскивании до ±12° (`rotationX` / `rotationY`), возврат пружиной.

107. Зона нажатия — вся плашка; жесты отдаются наружу через лямбды.

108. Долгое нажатие копирует номер в буфер (кириллицей или латиницей на выбор).

109. Экспорт PNG 2048/4096 px с прозрачным фоном и водяным знаком «Игровой номер».

110. Предпросмотр на авто: перспективное преобразование (`Matrix` / `Camera`) в точку крепления.

111. Тёмная тема не меняет цвета знака.

112. Семантика: `contentDescription` вида «Номер Р 206 УО, регион 07, Россия».

113. Скриншот-тесты (Paparazzi/Roborazzi) для всех типов знаков и размеров шрифта.

114. Отрисовка знака ⩽ 1 мс; ноль аллокаций внутри `DrawScope`.

115. Размеры `mini` / `small` / `medium` / `hero` — один компонент, один набор параметров.

116. Смена номера анимируется на месте, без пересоздания всего знака.

117. Тень знака — `drawBehind` с кэшем, а не `shadow()` на каждом элементе списка.

118. Подпись раскладки: кириллица или латиница (для не-RU локалей).

119. Переключатель «передний / задний» номер (у некоторых типов разные пропорции).

120. Режим «Классика»: визуальная совместимость со старым стилем прототипа.

Раздел 4. Генерация номера: слот-ролл и микроанимации (121–160)

121. Каждый символ — отдельный барабан с вертикальной прокруткой.

122. Остановка слева направо с шагом 120–180 мс; общая длительность 0,9–1,4 с.

123. Кривая: быстрый разгон, замедление (`CubicBezierEasing(0.2f, 0f, 0f, 1f)`), overshoot 4–6%.

124. Лента барабана содержит только допустимые символы, порядок перемешивается на каждый ролл.

125. Motion blur: вертикальный `RenderEffect` blur (API 31+), запасной вариант — штрихи.

126. Anticipation: перед стартом знак сжимается до 0.97 и пружинит обратно.

127. Результат вычислен и сохранён до анимации; анимация не может изменить исход.

128. Повторный тап ускоряет ролл ×3; двойной — показывает результат мгновенно.

129. Честный драматизм: замедление последних барабанов — только при реальной редкости от «Эпический»; без ложных «почти-выигрышей».

130. Раскрытие редкости: частицы цвета тира (Canvas), 40–200 штук в зависимости от уровня.

131. Лёгкая тряска экрана: амплитуда по тиру (0–6 dp), отключаемая.

132. Для «Легендарный» и выше: виньетка и zoom камеры 1.0→1.06.

133. Хаптик `TICK` на каждой остановке барабана; финальный паттерн по тиру (раздел 15).

134. Цена «набегает» за 400 мс (easeOutCubic) от 0 до значения.

135. Списание стоимости: монеты «вылетают» с баланса к кнопке.

136. Продажа: монеты летят от знака к балансу, баланс «подпрыгивает» (scale 1.1).

137. Во время ролла кнопка: «Крутим…» + кольцевой прогресс, disabled.

138. Автогенерация: скорости ×1 / ×2 / ×4 / Turbo; Turbo — без анимаций, только сводка.

139. Остановка автогенерации: по редкости ⩾ X, лимиту трат, нехватке средств или кнопкой «Стоп».

140. Сводка автогенерации: число прокруток, потрачено, лучшие находки, итог в ₽.

141. Idle: свечение кнопки «дышит» раз в 3 с; выключается при Reduce motion.

142. Reduce motion: ролл заменяется кросс-фейдом 150 мс.

143. Уход в фон посреди ролла: результат применяется мгновенно, анимация не «догоняет».

144. Пул частиц: массивы примитивов, ноль аллокаций в кадре.

145. На каждый барабан — `Animatable`; общее состояние поднято в стабильный `RollState`.

146. Параметры анимации — в `RollSpec` (длительности, easing, пружины), конфигурируемые.

147. Debug-панель тюнинга `RollSpec` в реальном времени (только debug).

148. Конфетти для «Легендарный», неоновые искры для «Мифический».

149. Найденные паттерны подписываются плашками над знаком («ТРОЙКА 777», «ЗЕРКАЛО») с задержкой 150 мс.

150. Номер из результата уменьшается и скользит в ленту последних находок.

151. «Заморозка» символа: замок на позиции, цена +X%, анимация защёлкивания.

152. «Как выпало»: показать seed и лог генерации — для прозрачности.

153. Звук синхронен барабанам: высота тона растёт к последнему.

154. Новый личный рекорд: золотая лента «НОВЫЙ РЕКОРД» с коротким джинглом.

155. Жест «потяни вниз — крутить» (pull-to-roll) (опц.).

156. Тесты анимаций: `mainClock.autoAdvance = false`, проверка промежуточных и финального состояний.

157. Профилирование: ролл не роняет кадр > 16 мс (Perfetto, `androidx.tracing`).

158. Анимации в `LaunchedEffect`; при уходе с экрана корутины отменяются, утечек нет.

159. Все длительности и кривые берутся из motion-токенов (раздел 19).

160. Кнопка «Пропустить» появляется после 1-й секунды ролла и не мешает при первом запуске.

Раздел 5. Прокрутка и «откидывание номеров вверх» (161–200)

161. В списках (лента, гараж, результаты) номера, уходящие за верхний край, откидываются вверх на «шарнире» по верхнему ребру.

162. `graphicsLayer { rotationX = …; transformOrigin = TransformOrigin(0.5f, 0f); cameraDistance = 12f * density }`.

163. Угол от позиции: `progress = ((viewportTop - itemTop) / itemHeight).coerceIn(0f, 1f)`, `rotationX = -90f * easeOutCubic(progress)`.

164. Позиция читается внутри лямбды `graphicsLayer` (фаза отрисовки) — без рекомпозиции.

165. Параллельно: `alpha` 1→0.15, `scale` 1→0.92, затемнение к 90°.

166. Тень под откинутым номером растёт и смещается (`drawBehind`).

167. Блик на знаке при угле 20–60°: градиент зависит от `rotationX`.

168. Входящие снизу номера «складываются» из +50° в 0° (опц., отдельный тумблер).

169. Интенсивность зависит от скорости прокрутки: максимальный угол 30°–90° (`NestedScrollConnection`).

170. Возврат при обратной прокрутке — пружина: `spring(dampingRatio = 0.7f, stiffness = 300f)`.

171. Стыки без дыр: элементы перекрываются на 8 dp, `zIndex` по индексу.

172. Одновременно откидываются не более 3–4 элементов, остальные просто гаснут по `alpha`.

173. Хаптик-тик при пересечении −45° (не чаще раза в 60 мс).

174. Звук «щёлк» при откидывании: высота тона зависит от скорости, rate-limit, отключаемый.

175. Настройка «Эффект откидывания»: Выкл / Мягкий / Сильный (0–100%); при Reduce motion — fade.

176. Липкие заголовки по редкости (`stickyHeader`); эффект к ним не применяется.

177. `key = plate.id` , `contentType = PlateRow` ; вложенных lazy-списков нет.

178. Режим «Карусель»: `SnapFlingBehavior` и центровка по элементу.

179. Overscroll: stretch на Android 12+; на старых — свой bounce.

180. Быстрая прокрутка: ползунок с бабблом (первая буква или цена).

181. Кнопка «Наверх» после 8 элементов (fade + scale); тап — плавный скролл.

182. Параллакс фона на 0.3× от скорости прокрутки.

183. Pull-to-refresh: обновление ленты или новая генерация.

184. `Modifier.animateItem()` для добавления, удаления и перестановки.

185. Свайп-действия: влево — продать, вправо — в избранное; порог 40%, хаптик на пороге.

186. Свайп-продажа отменяется через Snackbar «Отменить» 5 с.

187. Мультивыбор долгим тапом; панель действий выезжает снизу.

188. Перестановка drag-and-drop в «Витрине» (reorderable, хаптик).

189. Экран «Меню»: карточки наклоняются (`rotationY` ±6°) по расстоянию от центра, иконки двигаются с параллаксом.

190. Горизонтальный `Pager` фильтров редкости с индикатором-«таблеткой».

191. Позиция скролла сохраняется по вкладкам (`rememberLazyListState` + `rememberSaveable`).

192. Списки 10 000+ элементов: Paging 3 + Room, плейсхолдеры, без лагов.

193. Эффект считается только для видимых элементов (`layoutInfo.visibleItemsInfo`).

194. Формула — в одном месте: модификатор `PlateFlipEffect` с KDoc и формулой.

195. Скриншот-тесты на углах 0° / −45° / −90°.

196. Macrobenchmark: прокрутка 1000 элементов, janky frames < 2%.

197. TalkBack: порядок обхода сохраняется, доступны `scrollBy`-действия; эффект не влияет на фокус.

198. Слабое устройство (≤ 4 ГБ RAM): при просадке FPS эффект автоматически смягчается.

199. Debug-экран «Лаборатория скролла»: настройка угла, камеры, кривой и скорости.

200. В README — GIF или видео с демонстрацией откидывания и формулой.

Раздел 6. Редкость, паттерны и оценка стоимости (201–240)

201. Пять тиров: «Обычный», «Редкий», «Эпический», «Легендарный», «Мифический» — по числу сегментов шкалы.

202. Тир — данные (`enum`/конфиг): цвет, свечение, звук и хаптик берутся из токенов.

203. Детектор паттернов — набор чистых правил (`PatternRule`), расширяемый без правки ядра.

204. Правила цифр: тройки и повторы (`777`), последовательности (`123`/`321`), зеркала (`121`), «круглые» (`100`, `001`).

205. Правила букв: `AAA`, одинаковые буквы, читаемые сочетания из допустимых букв.

206. Премиальные регионы (например, 77, 99, 177, 197, 777, 799): множитель из таблицы данных.

207. Комбинации-«даты» (ДДММ, год) дают небольшой бонус.

208. Цена = база × тир × паттерны × регион × тип; формула и коэффициенты — в конфиге.

209. В UI — разбор цены списком бонусов (+x%) с названиями паттернов.

210. Таблица шансов по тирам (сумма 100%) показывается игроку в один тап.

211. Тест Монте-Карло на 10^6 генераций: фактическое распределение совпадает с теоретическим (χ^2).

212. Pity: гарантия «Редкий» и выше каждые N прокруток, «Эпический» и выше — каждые M; счётчик виден.

213. Pity-счётчик хранится в Room и не сбрасывается перезапуском.

214. Взвешенный выбор без смещения (alias-метод); seed — из `SecureRandom`.

215. Уникальность: повторный номер считается «дубликатом» и превращается в осколки.

216. Валидация формата по спецификации страны выполняется до показа игроку.

217. Маска поиска: `*` — любой символ, `#` — цифра, `@` — буква; используется в заданиях и фильтрах.

218. Рыночная цена колеблется ±10% в день (детерминированно по дате и seed), показывается стрелка тренда.

219. Шкала из 5 сегментов заполняется с «щелчками»; цвет сегментов — по тиру.

220. Рамка карточки: тонкая у «Обычного», анимированный градиент у «Мифического».

221. Тир обозначается иконкой и текстом, а не только цветом (доступность).

222. Для находки показывается шанс выпадения («Шанс: 0,12%»).

223. Экран «Статистика»: гистограмма ваших прокруток против теоретической.

224. Коллекционные сеты (все тройки, `AAA` – `XXX`, регионы) с прогрессом и наградой.

225. Карта регионов РФ: открытые и неоткрытые регионы (опц.).

226. Динамический спрос: частые продажи одного паттерна снижают цену, восстановление за 24 ч.

227. Потолок цены по тирам — защита от инфляции, настраивается в конфиге.

228. В каждом номере хранится `valuationVersion`; при смене правил переоценка показывается игроку.

229. Метаданные номера: seed, время, версия правил, режим генерации.

230. Debug-симулятор «10 000 прокруток» с отчётом по тирам и деньгам.

231. Правила — в JSON с валидацией схемы; горячая подмена через Remote Config.

232. Табличные unit-тесты на каждое правило, не менее 100 кейсов.

233. Каталог «Легенды»: мифические маски (например, `A777AA`) с подсказкой, как их получить.

234. Серия без редких даёт мягкий бонус к шансам — честно и с отображением в UI.

235. «Огранка»: слияние 5 дубликатов в улучшение тира (опц.).

236. Режим «Охота»: игрок задаёт маску, игра показывает шанс получить её за N попыток.

237. Сравнение: «ваш номер лучше, чем у X% найденных».

238. Свои таблицы шансов и множителей для каждой страны и типа номера.

239. Единый `RarityFormatter` для названий и склонений.

240. В аналитику пишутся тир, шанс, цена и pity-счётчик (без персональных данных).

Раздел 7. Страны и типы номеров (241–270)

241. Формат описывается декларативно в `PlateSpec` (шаблон, алфавит, цвета, пропорции, регионы), а не ветвлениями `if/else`.

242. Первая очередь стран: Россия, Беларусь, Казахстан, Украина, Армения, Узбекистан; форматы — по официальным стандартам.

243. Типы РФ: обычный, такси, прицеп, мото, ретро (СССР).

244. Служебные типы (дипломатический, военный, полицейский) — только игровые косметические версии с пометкой «Игровой».

245. Данные регионов — `regions.json` (код, название, страна, действует ли).

246. Выбор страны: bottom sheet с поиском, группами (СНГ, Европа, Азия, Америка), недавними и избранными.

247. Чип выбора: флаг + «Страна · Тип», анимация при смене.

248. Следующая генерация идёт строго в выбранном формате; текущий знак молча не подменяется.

249. Инвариант: страна и тип показанного знака совпадают с записью в БД; assert в debug и тест.

250. Страны открываются по уровню или за ₽; закрытые — силуэт и «Откроется на N уровне».

251. Разблокировка страны: флаг «разворачивается», короткий звук и хаптик.

252. Паттерны специфичны для страны (например, региональные коды) и живут в её `PlateSpec`.

253. У каждого типа свои `priceMultiplier` и `generationCost` в конфиге экономики.

254. Превью формата: пример номера и маска («Формат: A 000 AA 00»).

255. Алфавит ограничен по стране: для РФ — 12 кириллических букв, для остальных — по стандарту.

256. Транслитерация для латинских локалей — рядом со знаком (переключатель).

257. Тип «Ретро»: старый формат, свой шрифт и цвет (опц.).

258. Двухстрочные типы (мото, прицеп) — отдельный layout и размеры.

259. Правила отображения региона и флага задаются для каждой страны отдельно.

260. Коллекции по странам и типам с прогрессом («собрано 12 из 40»).

261. Фильтры и сортировка по стране и типу в гараже.

262. Экспортное изображение соответствует пропорциям и рамке типа.

263. Property-based тесты: сгенерированные номера всегда валидны по `PlateSpec`.

264. Скриншот-тесты: все типы рендерятся без обрезки при шрифте 1.0× и 2.0×.

265. Новая страна добавляется без изменения кода: только JSON и ассеты.

266. Названия стран и типов локализованы (ru/en и др.).

267. Режим «Песочница»: посмотреть любой формат бесплатно, без редкости и цены (опц.).

268. Рекомендация «Для вас: Казахстан» на основе коллекции (опц.).

269. Предупреждение при продаже последнего номера типа, нужного для сета.

270. `PlateType` хранится строковым ID, а не `ordinal`, — безопасно для миграций.

Раздел 8. Экономика и баланс (271–300)

271. Стоимость генерации считается по прозрачной формуле: на скриншоте 30 507 ₽ при балансе 20 138 013 ₽ (≈ 0,15%) — вынести в конфиг с минимумом и потолком.

272. В UI показать разбивку цены: база × режим × уровень.

273. Целевой RTP по режимам (например, 85–110%) задаётся в конфиге и проверяется симулятором.

274. Источники дохода: продажа, задания, гонки, ДПС, фортуна, события, пассивный доход.

275. Стоки: генерация, кейсы, улучшения, авто, штрафы, рамки; баланс источников и стоков отслеживается.

276. Леджер операций в Room: `reason`, `amount`, `balanceAfter`, `ts`.

277. Атомарность: списание и выдача — в одной транзакции, операции идемпотентны.

278. Формат чисел: «20 138 013 ₽» с неразрывными пробелами; компактно — «20,1 млн ₽».

279. Числа до триллионов и выше: `Long` / `BigDecimal`, защита от переполнения.

280. Пассивный доход «Парковка»: ₽/час, офлайн-лимит 8–12 ч, кнопка «Собрать» с анимацией.

281. Защита от подкрутки времени: монотонные часы + серверное время.

282. Страховка от тупика: при балансе ниже минимальной генерации — бесплатная попытка или задание-«пособие».

283. «История операций»: фильтры по типу и датам, экспорт в CSV.

284. Бустеры (например, ×2 доход) с таймерами; правила стекирования описаны и протестированы.

285. Комиссия продажи 5%, снижается улучшениями.

286. Рыночные модификаторы дня (по паттернам) показываются в UI.

287. Престиж («Перезапуск»): сброс прогресса за постоянный множитель и эксклюзивные скины.

288. Капитал = баланс + оценка коллекции; показывается в профиле.

289. Экономика — в `economy.json` с версией и схемой; обновление через Remote Config.

290. Симулятор экономики: 10 000 игроков × 30 дней, отчёт запускается в CI.

291. Сигнал об инфляции: медиана баланса растёт быстрее X% в неделю.

292. Суточный лимит дохода от заданий.

293. Игровая валюта только виртуальная; вывод невозможен; реальные платежи по умолчанию отсутствуют.

294. Если платежи появятся — биллинг магазинов (Google Play / RuStore) через абстракцию и с лимитами.

295. Неденежные валюты («ключи кейсов», «осколки») — отдельные счета в леджере.

296. Подтверждение операций, превышающих 10% баланса.

297. Сводка сессии при выходе: потрачено, заработано, лучшая находка.

298. Деньги отмечаются цветом и знаком (+/−), а не только цветом.

299. Суммы — целые ₽; отрицательные значения и переполнение отсекаются валидацией.

300. Property-based тесты инвариантов: сумма леджера равна балансу.

Раздел 9. Коллекция, гараж и инвентарь (301–340)

301. Экран «Гараж»: переключатель «сетка / список» с анимацией.

302. Сортировка: цена, редкость, дата, регион, алфавит.

303. Фильтры-чипы: тир, страна, тип, регион, избранное, дубликаты, маска.

304. Поиск по номеру с подсветкой совпадений; FTS в Room; результат мгновенный.

305. Сохраняемые пресеты фильтров.

306. «Избранное» и «Замок»: защита от продажи.

307. Массовая продажа: выбор, итог, подтверждение; защищённые номера исключаются.

308. Карточка номера: крупный знак, тир, разбор цены, история цены, дата и seed.

309. Действия: «Продать», «Установить на авто», «Поделиться», «В витрину», «Заметка».

310. «Витрина»: топ-6 номеров профиля, перестановка перетаскиванием.

311. Парковочные слоты: ёмкость, расширение за ₽; при переполнении — авто-продажа по правилам игрока.

312. Дубликаты сливаются в осколки; перед слиянием — предпросмотр.

313. Сеты и коллекции с кольцевым прогрессом и наградой за завершение.

314. Бейдж NEW до первого просмотра номера.

315. Пустое состояние: иллюстрация и кнопка «Сгенерировать».

316. Корзина на 24 ч (soft delete) с восстановлением.

317. Flip-карточка: переворот (`rotationY` 180°) показывает детали на обороте.

318. Сравнение двух номеров side-by-side.

319. График капитала коллекции (Canvas) за 24 ч / 7 д / 30 д.

320. Пользовательские теги и заметки.

321. Липкие заголовки по тиру или стране при группировке.

322. Режимы просмотра: «Плитка», «Лента», «Карусель» (3D coverflow).

323. Coverflow: центральный знак крупно, боковые под углом 35° (`graphicsLayer`).

324. Пагинация Paging 3 и индексы БД рассчитаны на 100 000+ номеров.

325. Оптимистичные обновления UI: действие видно мгновенно, откат при ошибке.

326. Ленивая подгрузка рендеров знаков через кэш.

327. `animateItem` при продаже и добавлении.

328. Мультивыбор: панель выезжает снизу, счётчик выбранных.

329. Продажа редких (тир от «Эпический») — дополнительный шаг подтверждения.

330. «Лучшие находки» закреплены наверху.

331. Статистика коллекции: количество и стоимость по тирам (диаграмма).

332. Подсказка «Подходит в сет X» на карточке номера.

333. Экспорт и импорт коллекции (JSON/CSV).

334. Резервное копирование коллекции — см. раздел «Данные, сохранения и облако».

335. Установка номера на авто: передний/задний слот; бонус за «комплект».

336. «Паспорт» авто и номера: имя, история, пробег в гонках.

337. Доступность: выбор элементов озвучивается («Выбрано 3 из 20»).

338. Скриншот-тесты сетки при трёх масштабах шрифта.

339. Прокрутка 5000 элементов держит 60 fps на среднем устройстве.

340. Журнал изменений коллекции берётся из леджера (кто, что, когда).